

This guide explains how to use the checklist table token in reports using a query string-like syntax.

BASIC STRUCTURE

The token uses URL-style parameters separated by `&` characters. For example:

```
{Checklist_Table?
columns=code,name,status,findings&status=Todo,InProgress&
setting.sort.field=status&setting.sort.direction=Ascending}
```

CONFIGURATION SECTIONS

1. REQUIRED PARAMETERS

These are the required parameters and their options:

columns=[field1,field2,...]

Available field names:

- `code` - Task code/identifier
- `name` - Task name/title
- `description` - Task description
- `status` - Task status (Todo, InProgress, Skipped, Done)
- `assigned_to` - Person assigned to the task
- `reporter` - Person who reported/created the task
- `findings` - Associated findings by severity (shows badges with counts)
- `assets` - Associated assets

Default columns (if not specified): `code, name, status, findings`

2. FILTERING (OPTIONAL)

Filter checklist tasks by various criteria:

checklist=[ChecklistName]

- Filter by specific checklist name
- Case-insensitive exact match
- Example: `checklist=OWASP ASVS`

status=[Status1,Status2,...]

- Filter by task statuses
- Available values: `Todo`, `InProgress`, `Skipped`, `Done`
- Multiple values use OR logic
- Example: `status=Todo,InProgress`

groups=[Group1,Group2,...]

- Filter by checklist group codes
- Case-insensitive matching
- Example: `groups=V1,V2,V3` (for OWASP ASVS verification categories)

assets=[Asset1, Asset2,...]

- Filter by associated assets
- Matches against asset Title, URL, IP, or Repository
- Case-insensitive partial match
- Multiple values use OR logic
- Example: `assets=192.168.1.1,webapp.example.com`

severity=[Severity1,Severity2,...]

- Filter by finding severity
- Available values: Critical, High, Medium, Low, Info
- Case-insensitive
- Multiple values use OR logic
- Only shows tasks that have findings with the specified severities
- Example: `severity=Critical,High`

assigned_to=[name]

- Filter by assigned user
- Case-insensitive partial match
- Example: `assigned_to=John`

3. SORTING (OPTIONAL)

Sort checklist tasks by a specific field:

setting.sort.field=[fieldname]

- Available fields: `code`, `name`, `status`, `assigned_to`, `reporter`, `findings`, `assets`
- Default: `code`

setting.sort.direction=[Ascending|Descending]

- Sort direction
- Available values: `ascending`, `descending`
- Default: `ascending`

HTML STRUCTURE AND CSS CLASSES

The Checklist Table token generates tabular data organized by checklist groups with configurable columns.

ROOT CONTAINER

```
<div class="checklist-table-container">
```

Purpose: Top-level container for all table content

CSS Classes:

- `checklist-table-container`: Base wrapper class

CHECKLIST HEADERS (WHEN MULTIPLE CHECKLISTS)

```
<h2 class="checklist-table-header">Checklist Name</h2>
```

Purpose: Displays the checklist name when multiple checklists are present

CSS Classes:

- `checklist-table-header`: Checklist name heading

GROUP HEADERS

```
<h3 class="checklist-group-header">Group Code | Group Name</h3>
```

Purpose: Displays the group identifier and name (e.g., "V1 | Architecture, Design and Threat Modeling")

CSS Classes:

- `checklist-group-header`: Group heading

TABLE STRUCTURE

```
<table class="checklist-task-table">
  <thead>
    <tr>
      <th class="table-header [column-name]">Column Display
Name</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td class="table-cell [column-name]">Cell Content</td>
    </tr>
  </tbody>
</table>
```

CSS Classes:

- `checklist-task-table` : Base table styling
- `table-header` : Header cell base class
- `table-cell` : Data cell base class
- `[column-name]` : Column-specific class
(e.g., `code`, `name`, `status`, `findings`, `assets`)

STATUS BADGES

```
<span class="kt-badge kt-badge--inline status-task-[status]">Status
Label</span>
```

CSS Classes:

- `kt-badge kt-badge--inline` : Base badge classes
- `status-task-todo` : Todo status
- `status-task-inprogress` : In Progress status
- `status-task-skipped` : Skipped status
- `status-task-done` : Done status

FINDING SEVERITY BADGES

```
<span class="kt-badge kt-badge--inline criticality-finding-
[severity]"
  style="margin-bottom: 1px; margin-right: 2px;">
  Severity (Count)
</span>
```

CSS Classes:

- `kt-badge kt-badge--inline` : Base badge classes
- `criticality-finding-critical` : Critical severity
- `criticality-finding-high` : High severity
- `criticality-finding-medium` : Medium severity
- `criticality-finding-low` : Low severity
- `criticality-finding-info` : Info severity

EXAMPLES

BASIC TABLE WITH DEFAULT COLUMNS

```
{Checklist_Table}
```

This displays all checklist tasks with default columns: code, name, status, and findings.

TABLE FILTERED BY CHECKLIST AND STATUS

```
{Checklist_Table?checklist=OWASP ASVS&status=Todo,InProgress}
```

Shows only tasks from the OWASP ASVS checklist that are not yet completed (Todo or InProgress).

COMPREHENSIVE TASK VIEW WITH ALL DETAILS

```
{Checklist_Table?  
columns=code,name,description,status,assigned_to,findings,assets&  
setting.sort.field=status&setting.sort.direction=Ascending}
```

Displays a complete view of all tasks with code, name, description, status, assignee, findings, and assets, sorted by status.

OUTSTANDING TASKS REPORT

```
{Checklist_Table?columns=name,status,assigned_to,findings&  
status=Todo,InProgress&  
setting.sort.field=findings&  
setting.sort.direction=Descending}
```

Focuses on incomplete tasks, showing who's responsible and associated findings, sorted by finding count (highest first).